Name: Colton Zachary

UCO Email: czachary@uco.edu

Project Number: p01

Server Username: x1025

1. Which version of the list implementations is better: contiguous or linked?

The linked list is better for this project because

It allows efficient insertions anywhere without shifting elements O(1) if position is known.

Memory is allocated dynamically, so it does not have a fixed maximum size like the array-based list (max_list).

The contiguous (array) list is simpler and can be slightly faster for traversal because elements are stored

sequentially in memory, which is cache-friendly.

Trade-off: For frequent sorted insertions and unknown list sizes, linked lists are more flexible.

For small, fixed-size lists, contiguous arrays are slightly faster.

2. How to understand the function void traverse(void (*visit)(List_entry &))?

traverse iterates through every element in the list and applies a function pointer visit to each element.

For example, if visit is print_record, it will print all records in the list.

This design decouples list storage from operations on elements, making the list reusable for different

tasks (printing, processing, modifying).

3. What’s the search algorithm you used in Task 3? Why is it suitable?

Algorithm: Linear search

Loop through the list from start to end.

For ID lookup, check if record.ID == searchID.

For name prefix lookup, check if first_name and last_name start with the given prefix.

Why suitable:

The list is sorted, but the search is for prefix matches or exact matches, not a full ordering.

Linear search guarantees that all matches are found without relying on more complex structures.

It works correctly for both linked and array implementations.

4. Can you directly use comparison operators (>, <, >=, etc.) on Personal_record instances?

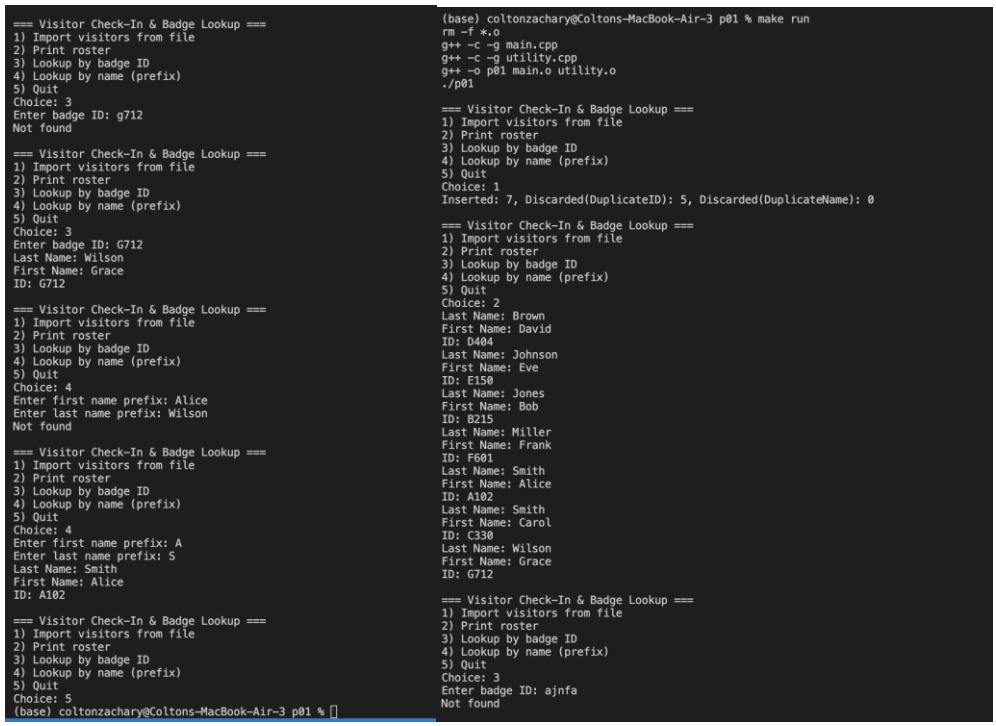
No, because C++ does not know how to compare two Personal_record structs automatically.

Solution used: Implemented a comparison function "comes_before".

This function defines ordering based on last name → first name → ID.

We use it to maintain sorted order in the list and during insertion.

5. Screenshot of a Test Run



```
=== Visitor Check-In & Badge Lookup ===
1) Import visitors from file
2) Print roster
3) Lookup by badge ID
4) Lookup by name (prefix)
5) Quit
Choice: 3
Enter badge ID: g712
Not found

=== Visitor Check-In & Badge Lookup ===
1) Import visitors from file
2) Print roster
3) Lookup by badge ID
4) Lookup by name (prefix)
5) Quit
Choice: 3
Enter badge ID: G712
Last Name: Wilson
First Name: Grace
ID: G712

=== Visitor Check-In & Badge Lookup ===
1) Import visitors from file
2) Print roster
3) Lookup by badge ID
4) Lookup by name (prefix)
5) Quit
Choice: 4
Enter first name prefix: Alice
Enter last name prefix: Wilson
Not found

=== Visitor Check-In & Badge Lookup ===
1) Import visitors from file
2) Print roster
3) Lookup by badge ID
4) Lookup by name (prefix)
5) Quit
Choice: 4
Enter first name prefix: A
Enter last name prefix: S
Last Name: Smith
First Name: Alice
ID: A102

=== Visitor Check-In & Badge Lookup ===
1) Import visitors from file
2) Print roster
3) Lookup by badge ID
4) Lookup by name (prefix)
5) Quit
Choice: 5
(base) coltonzachary@Coltons-MacBook-Air-3 p01 %
```

```
(base) coltonzachary@Coltons-MacBook-Air-3 p01 % make run
rm -f *.o
g++ -c -g main.cpp
g++ -c -g utility.cpp
g++ -o p01 main.o utility.o
./p01

=== Visitor Check-In & Badge Lookup ===
1) Import visitors from file
2) Print roster
3) Lookup by badge ID
4) Lookup by name (prefix)
5) Quit
Choice: 1
Inserted: 7, Discarded(DuplicateID): 5, Discarded(DuplicateName): 0

=== Visitor Check-In & Badge Lookup ===
1) Import visitors from file
2) Print roster
3) Lookup by badge ID
4) Lookup by name (prefix)
5) Quit
Choice: 2
Last Name: Brown
First Name: David
ID: D404
Last Name: Johnson
First Name: Eve
ID: E150
Last Name: Jones
First Name: Bob
ID: B215
Last Name: Miller
First Name: Frank
ID: F601
Last Name: Smith
First Name: Alice
ID: A102
Last Name: Smith
First Name: Carol
ID: C330
Last Name: Wilson
First Name: Grace
ID: G712

=== Visitor Check-In & Badge Lookup ===
1) Import visitors from file
2) Print roster
3) Lookup by badge ID
4) Lookup by name (prefix)
5) Quit
Choice: 3
Enter badge ID: ajnfa
Not found
```

6. main.cpp Source File

```
#include "utility.h"


#include <fstream>

#include <iostream>

#include <limits>

#include <set>

#include <string>


#include "Node.h"

// #include "List2.h"

#include "List1.h" // defines template List<T>


static void clear_line() {
```

```
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

static void print_roster(List<Personal_record> &roster) {
// Fill in your code
roster.traverse(print_record);
}

static void lookup_by_id(const List<Personal_record> &roster) {
// Fill in your code
// You might need to implement and use find_by_id in utility.h
std::string id;
std::cout << "Enter badge ID: ";
std::cin >> id;
clear_line();

Personal_record result;
if (find_by_id(roster, id, result)) {
   print_record(result);
} else {
   std::cout << "Not found\n";
}
}

static void lookup_by_name_prefix(const List<Personal_record> &roster) {
// Fill in your code
// You might need to implement and use find_all_by_name_prefix in utility.h
std::string first, last;
std::cout << "Enter first name prefix: ";
```

```
std::cin >> first;
std::cout << "Enter last name prefix: ";
std::cin >> last;
clear_line();

int matches = find_all_by_name_prefix(roster, first, last, print_record);
if (matches == 0) {
   std::cout << "Not found\n";
}
}

static void menu() {
std::cout << "\n=== Visitor Check-In & Badge Lookup ===\n"
          << "1) Import visitors from file\n"
          << "2) Print roster\n"
          << "3) Lookup by badge ID\n"
          << "4) Lookup by name (prefix)\n"
          << "5) Quit\n"
          << "Choice: ";
}

int main() {
List<Personal_record> roster;

while (true) {
   menu();

   int choice;
   if (!(std::cin >> choice)) {
```

```
// EOF or invalid input
return 0;
}
clear_line();

switch (choice) {
case 1: {
// fill in your code to implement task 1
Import_stats stats = import_file(roster, "data.txt");

std::cout << "Inserted: " << stats.inserted
          << ", Discarded(DuplicateID): " << stats.duplicate_id
          << ", Discarded(DuplicateName): " << stats.duplicate_name
          << std::endl;
break;
}
case 2:
print_roster(roster); // Implement this function
break;
case 3:
lookup_by_id(roster); // Implement this function
break;
case 4:
lookup_by_name_prefix(roster); // Implement this function
break;
case 5:
return 0;
default:
std::cout << "Invalid choice.\n";
break;
```

```
    }
}
}
```